软件质量保障 选题汇报

—KNighter：利用LLM合成的静态分析检查器

小组：题目5小组3

选题基本情况

PART ONE

选题基本情况

01 项目简介

—KNighter 是一款创新的检查综合工具，利用大型语言模型（LLM）自动从历史补丁提交生成静态分析检查器。

—主要特征：

LLM驱动生成：利用最先进的语言模型自动合成静态分析检查器

多步骤流程：采用复杂的生成→精细化→分流工作流程，实现高质量结果

历史学习：通过真实补丁提交学习，理解常见的漏洞模式

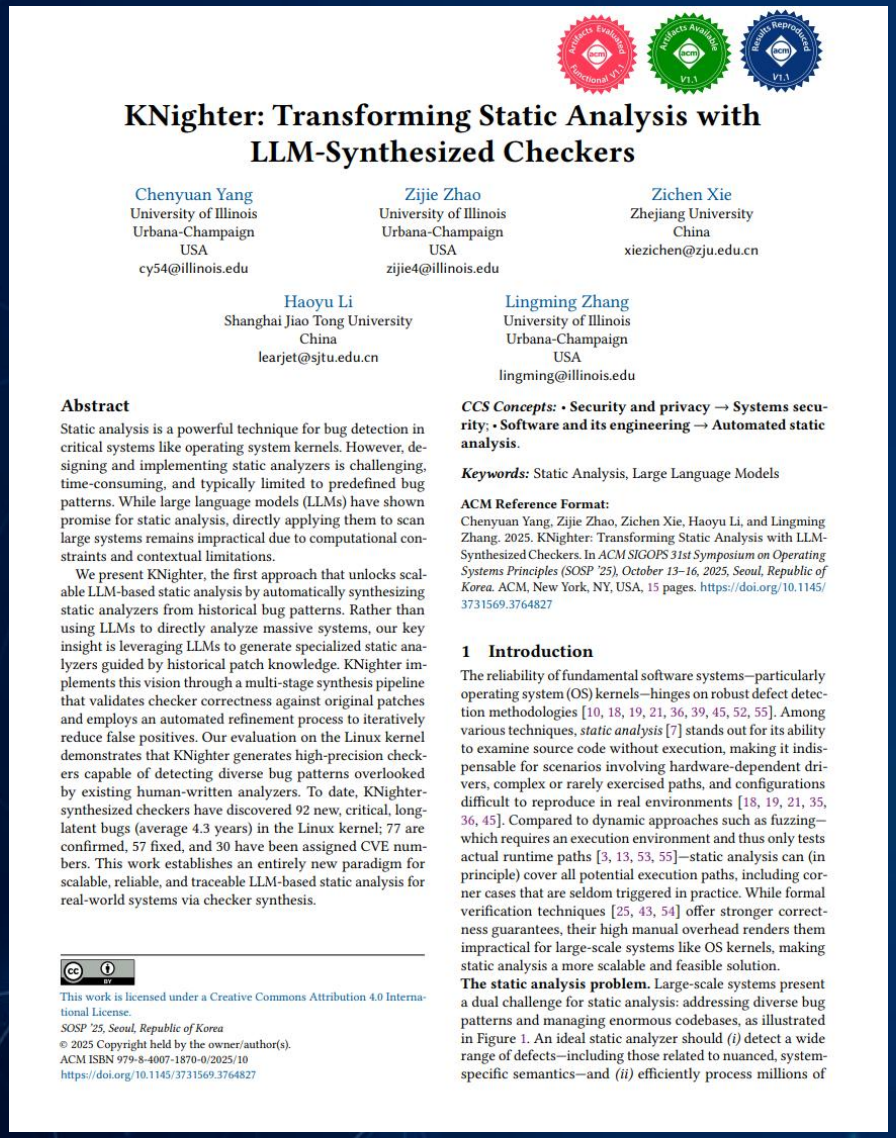
LLVM 集成：基于 LLVM 构建，提供强大的静态分析能力

Linux 内核焦点：专门用于发现大规模 C/C++ 代码库（如 Linux 内核）中的漏洞。

—项目链接：https://github.com/ise-uiuc/KNighter

—有关调研论文：Chenyuan Yang, Zijie Zhao, Zichen Xie, Haoyu Li, and Lingming Zhang. 2025. KNighter: Transforming Static Analysis with LLM-Synthesized Checkers. In ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25), October 13–16, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3731569.3764827

选题基本情况



KNighter: Transforming Static Analysis with LLM-Synthesized Checkers

Chenyuan Yang
University of Illinois
Urbana-Champaign
USA
cy54@illinois.edu

Zijie Zhao
University of Illinois
Urbana-Champaign
USA
zijie4@illinois.edu

Zichen Xie
Zhejiang University
China
xiezichen@zju.edu.cn

Haoyu Li
Shanghai Jiao Tong University
China
learjet@sjtu.edu.cn

Lingming Zhang
University of Illinois
Urbana-Champaign
USA
lingming@illinois.edu

Abstract

Static analysis is a powerful technique for bug detection in critical systems like operating system kernels. However, designing and implementing static analyzers is challenging, time-consuming, and typically limited to predefined bug patterns. While large language models (LLMs) have shown promise for static analysis, directly applying them to scan large systems remains impractical due to computational constraints and contextual limitations.

We present KNighter, the first approach that unlocks scalable LLM-based static analysis by automatically synthesizing static analyzers from historical bug patterns. Rather than using LLMs to directly analyze massive systems, our key insight is leveraging LLMs to generate specialized static analyzers guided by historical patch knowledge. KNighter implements this vision through a multi-stage synthesis pipeline that validates checker correctness against original patches and employs an automated refinement process to iteratively reduce false positives. Our evaluation on the Linux kernel demonstrates that KNighter generates high-precision checkers capable of detecting diverse bug patterns overlooked by existing human-written analyzers. To date, KNighter-synthesized checkers have discovered 92 new, critical, long-latent bugs (average 4.3 years) in the Linux kernel; 77 are confirmed, 57 fixed, and 30 have been assigned CVE numbers. This work establishes an entirely new paradigm for scalable, reliable, and traceable LLM-based static analysis for real-world systems via checker synthesis.

This work is licensed under a Creative Commons Attribution 4.0 International License.

SOSP '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1870-0/2025/10
https://doi.org/10.1145/3731569.3764827

CCS Concepts: • **Security and privacy → Systems security**; • **Software and its engineering → Automated static analysis**.

Keywords: Static Analysis, Large Language Models

ACM Reference Format:
Chenyuan Yang, Zijie Zhao, Zichen Xie, Haoyu Li, and Lingming Zhang. 2025. KNighter: Transforming Static Analysis with LLM-Synthesized Checkers. In *ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25), October 13–16, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3731569.3764827

1 Introduction

The reliability of fundamental software systems—particularly operating system (OS) kernels—hinges on robust defect detection methodologies [10, 18, 19, 21, 36, 39, 45, 52, 55]. Among various techniques, *static analysis* [7] stands out for its ability to examine source code without execution, making it indispensable for scenarios involving hardware-dependent drivers, complex or rarely exercised paths, and configurations difficult to reproduce in real environments [18, 19, 21, 35, 36, 45]. Compared to dynamic approaches such as fuzzing—which requires an execution environment and thus only tests actual runtime paths [3, 13, 53, 55]—static analysis can (in principle) cover all potential execution paths, including corner cases that are seldom triggered in practice. While formal verification techniques [25, 43, 54] offer stronger correctness guarantees, their high manual overhead renders them impractical for large-scale systems like OS kernels, making static analysis a more scalable and feasible solution.

The static analysis problem. Large-scale systems present a dual challenge for static analysis: addressing diverse bug patterns and managing enormous codebases, as illustrated in Figure 1. An ideal static analyzer should *(i)* detect a wide range of defects—including those related to nuanced, system-specific semantics—and *(ii)* efficiently process millions of

调研论文展示

摘要

静态分析是一种在操作系统内核等关键系统中检测缺陷的强大技术。然而，设计和实现静态分析器既困难又耗时，且通常仅限于预定义的缺陷模式。尽管大型语言模型（LLMs）在静态分析领域展现出巨大潜力，但由于计算资源的限制和上下文的局限性，直接将其应用于扫描大型系统仍不切实际。

我们推出KNighter，这是首个通过从历史错误模式中自动合成静态分析器，从而实现基于大型语言模型（LLM）的可扩展静态分析的方法。与直接利用LLM分析庞大系统不同，我们的核心思路是借助LLM，在历史补丁知识的指导下生成专门的静态分析器。 KNighter 通过多阶段合成管道实现这一愿景：该管道会参照原始补丁验证检查器的正确性，并采用自动精炼流程来迭代减少误报。我们在 Linux 内核上的评估表明，KNighter 生成的检查器具有高精度，能够检测到现有人工编写分析器所忽略的各种缺陷模式。 迄今为止，KNighter 合成的检查器已在 Linux 内核中发现了 92 个新的、关键的、潜伏时间较长的漏洞（平均潜伏期为 4.3 年）；其中 77 个已得到确认，57 个已修复，30 个已被分配 CVE 编号。本研究通过检查器合成，为针对现实世界系统的可扩展、可靠且可追溯的基于 LLM 的静态分析确立了一个全新的范式。

选题基本情况

02 项目评估

—成熟度评估：项目成熟，相关论文已于2025年10月12日完成发表

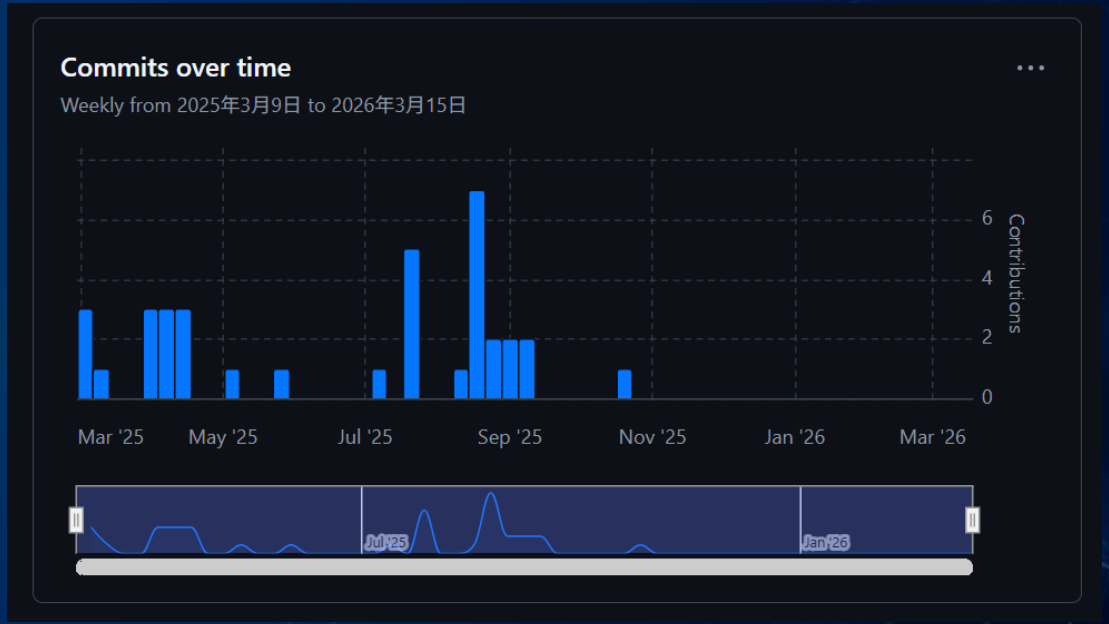
03 选择理由

—项目与大作业方向（5、LLM辅助Linux内核缺陷静态检测）完美契合

—主要使用语言为C++、Python，较为基础，并且项目环境的配置较为轻松

— 论文发表于SOSP'25（CCF A类会议），含金量高

—论文提到：迄今为止，基于KNighter合成的检查器已在Linux内核中发现了92个新的、关键且长期潜伏的漏洞，工具本身已发展成熟



初步改进思路

PART TWO

初步改进思路

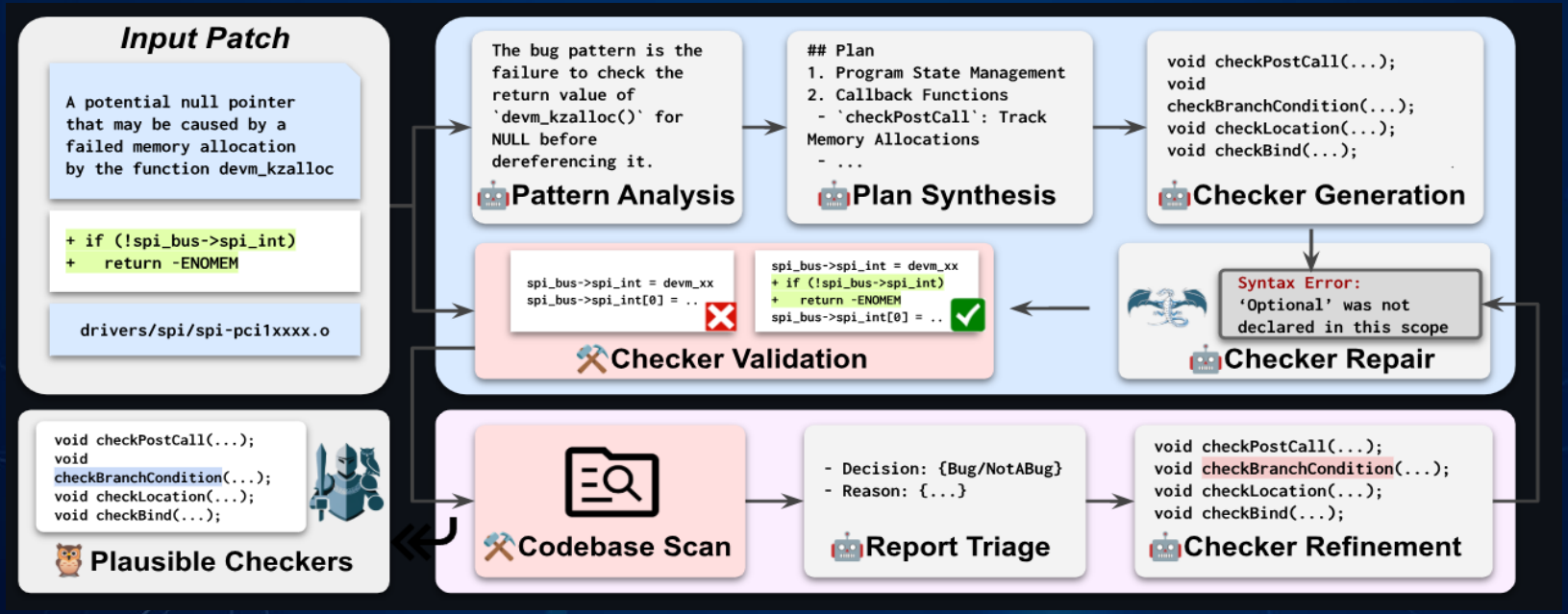
01 初步计划

我们初步计划从三个递进方向对Knighter进行改进：

—完成环境的部署与Knighter的使用，并对工具的整体架构进行探索与优化；

—针对LLM核心组件，开展不同API在检查器合成任务上的准确率与成本对比实验，通过量化评估选择最优模型组合，为实际应用提供性价比最佳的方案；

—深入优化LLM提示词设计，针对缺陷模式分析、计划合成和代码实现等关键环节设计更精准的提示模板，进一步提升合成检查器的有效性。



感谢观看

THANKS